



[WebII lab sheet] Student Version

[2nd Semester] [2025 - 2026]

ASP.NET Core 8 Minimal API Lab Manual

Instructor

Tiba Zaki Abdulhameed, Ph.D.

Lecturer Fathi Saad

Lecturer Assistant Ruaa Abdullah

March 2, 2026

Email

Tiba.zaki@nahrainuniv.edu.iq

Hours

[2 Hours]

From HTML to Service: Building a Bilingual Greeting App with ASP.NET Core 8 Minimal API

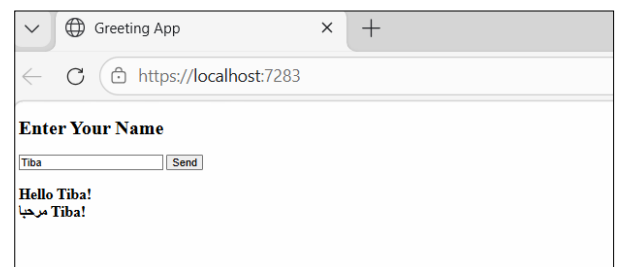
Learning Objectives

- Understand the role of Program.cs
- Create and register services
- Implement an interface with multiple implementations
- Use Dependency Injection
- Serve static HTML files
- Send requests from HTML to a Minimal API
- Return JSON responses
- Understand client-server communication

Project Story

We are building a simple web application:

1. The user enters their name in a web page.
2. The browser sends the name to the server.
3. The server generates English and Arabic greetings using services.
4. The server returns JSON.
5. The browser displays both greetings.



Final Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddScoped<EnglishGreetingService>();
```

```
builder.Services.AddScoped<ArabicGreetingService>();
```

```
var app = builder.Build();
```

```
app.UseDefaultFiles();
```

```
app.UseStaticFiles();
```

```
app.MapGet("/greet",
```

```
(string name, EnglishGreetingService english, ArabicGreetingService arabic) =>
```

```
{
```

```
    var enMessage = english.Greet(name);
```

```
    var arMessage = arabic.Greet(name);
```

```
    return new
```

```
{
```



[WebII lab sheet] Student Version

English = enMessage,
[2nd Semester] [2025 - 2026]
Arabic = arMessage

[Lab 3]
Instructor

Tiba Zaki Abdulhameed, Ph.D.
Lecturer Farah Saad
Lecturer Assistant Ruaa Abdullah

[March 2, 2026]

Email

Tiba.zaki@nahrainuniv.edu.iq

Hours

[2 Hours]

```
app.Run();
```

```
public interface IGreetingService  
{  
    string Greet(string name);  
}
```

```
public class EnglishGreetingService : IGreetingService  
{  
    public string Greet(string name)  
    {  
        return $"Hello {name}!";  
    }  
}
```

```
public class ArabicGreetingService : IGreetingService  
{  
    public string Greet(string name)  
    {  
        return $"مرحبا {name}!";  
    }  
}
```

HTML File (wwwroot/index.html)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Greeting App</title>
```

```
</head>
```

```
<body>
```

```
<h2>Enter Your Name</h2>
```

```
<input type="text" id="nameInput" placeholder="Your name">
```

```
<button onclick="sendName()">Get Greeting</button>
```

```
<h3 id="result"></h3>
```



[WebII lab sheet] Student Version

```
<script>
[2nd Semester] [2025 - 2026]
[Lab 3]
async function sendName() {
  const name = document.getElementById("nameInput").value;
  const response = await fetch("/greet?name=" + name);
  const data = await response.json();

  document.getElementById("result").innerHTML =
    data.English + "<br>" + data.Arabic;
}
</script>

</body>
</html>
```

[March 2, 2026]
Email

Hours

[2 Hours]

Student Task

Add a new service that calculates the length of the name.

1. Create an interface for name length.
2. Implement the service.
3. Register it.
4. Inject it into the endpoint.
5. Return the name length in JSON.
6. Display it in HTML.